



GUIA DEFINITIVO

A BÍBLIA DO BEAVERSEC

Comandos de terminal, scripts e configurações

B.E.A.V.E.R. – Binary Enumeration &
Advanced Vulnerability Exploitation Recon

v0.2.0

python 3.8+

MIT license

Versão 2.0.0 · Autor BeaverSec Team

Última atualização 20/06/2026 – noite

github.com/ojhonatanls/beaversec

Índice

01	Introdução
02	Status do projeto (leia antes de usar)
03	Estrutura do projeto
04	Pré-requisitos
05	Formas de execução
06	Comandos básicos
07	Módulos disponíveis
08	Erros comuns e soluções
09	Scripts prontos para uso
10	Exemplos práticos
11	Lista de comandos por tarefa
12	Perguntas frequentes
13	Contribuindo

01 Introdução

BeaverSec (B.E.A.V.E.R. — *Binary Enumeration & Advanced Vulnerability Exploitation Recon*) é um canivete suíço modular para cibersegurança, construído em Python com foco em aprendizado prático e evolução contínua.

Características principais

- ✓ **Modular:** cada ferramenta é um módulo independente
- ✓ **Leve:** os módulos atuais usam apenas bibliotecas padrão do Python
- ✓ **Multiplataforma:** funciona no Linux, Windows e macOS
- ✓ **Extensível:** fácil adicionar novos módulos
- ✓ **Educacional:** perfeito para aprendizado de cybersecurity

Este guia foi atualizado para refletir o estado real do código no repositório em 20/06/2026, após as últimas alterações no `main.py` e na estrutura de pastas. A seção 02 explica exatamente o que funciona hoje no terminal e o que ainda está em ajuste.

02 Status do projeto (leia antes de usar)

► POR QUE ESTA SEÇÃO EXISTE

O BeaverSec está em meio a uma migração: o ponto de entrada (`main.py`) ganhou uma CLI nova com subcomandos (`list` / `run`), mas nem todas as peças do projeto foram atualizadas para acompanhar essa mudança ainda. Esta seção resume o que é seguro usar hoje.

O que funciona sem erros

Item	Situação
<code>python3 main.py list</code>	OK lista os módulos (texto fixo no código)
Forma legada <code>module.run(target, verbose=...)</code> chamada direto via Python	OK é como <code>ping_sweep.py</code> e <code>port_scanner.py</code> funcionam de fato hoje
<code>ping_sweep</code> e <code>port_scanner</code> (lógica interna)	OK imprimem o resultado direto no terminal

Bugs conhecidos

△ BUG 1 – `python3 main.py run <módulo> --target <alvo>`

O novo subcomando `run` espera que cada módulo **retorne um dicionário** (ele chama `result.items()` internamente). Os módulos `ping_sweep` e `port_scanner` ainda retornam `None` (eles só imprimem na tela). Resultado: `AttributeError: 'NoneType' object has no attribute 'items'`.

△ BUG 2 – exportação em HTML (`--format html`)

`main.py` importa `beaversec.exporters.html_exporter`, mas o arquivo real está em `exporters/html_exporter.py`, fora do pacote `beaversec/`. Resultado: `ModuleNotFoundError: No module named 'beaversec.exporters'`.

△ BUG 3 – testes automáticos (`tests/test_modules.py`)

Os testes importam `beaversec.utils.security` (arquivo ainda não existe) e tentam importar módulos como `dns_enum`, `ssl_scan`, `http_headers`, `subdomain_brute` e `traceroute`, que aparecem listados em `main.py list` mas ainda não foram criados. Rodar `python3 -m unittest` falha hoje.

O que documentar como oficial

Seguindo a decisão do time: este guia documenta **as duas formas** de uso — a CLI nova (`list` / `run`), que é o caminho oficial e futuro, com aviso claro de bug em cada ponto onde ela quebra; e a forma direta/legada, que é a que funciona sem erros hoje para `ping_sweep` e `port_scanner`. Os scripts `.sh` (`setup.sh`, `scan.sh`, `install.sh`, `update.sh`) foram **atualizados nesta versão da Bíblia** para a sintaxe nova.

03 Estrutura do projeto

Estrutura real do repositório, conferida a partir do código-fonte atual. Note que `exporters/` fica na raiz, fora de `beaversec/` — é a causa do Bug 2 acima.

```
~/BEAVERSEC/
├── .github/ # Templates do GitHub
│   ├── ISSUE_TEMPLATE.md
│   └── PULL_REQUEST_TEMPLATE.md
├── beaversec/ # Pacote principal
│   ├── core/
│   │   ├── __init__.py
│   │   └── handler.py # ModuleHandler (não usado pelo main.py atual)
│   ├── modules/
│   │   ├── __init__.py
│   │   ├── ping_sweep.py # Varredura ICMP
│   │   ├── port_scanner.py # Varredura de portas
│   │   └── meu_modulo.py # Módulo de exemplo
│   ├── utils/
│   │   ├── __init__.py
│   │   └── logger.py # Sistema de logging
│   └── __init__.py
├── exporters/ # NOVO · fora de beaversec/, ver Bug 2
│   └── html_exporter.py
├── tests/
│   ├── __init__.py
│   └── test_modules.py # NOVO · falha hoje, ver Bug 3
├── .gitignore
├── BIBLIA_BEAVERSEC.md # Guia em markdown
├── Biblia_do_BeaverSec.pdf # Este guia em PDF
├── README.md
├── main.py # NOVO · CLI com list / run, ver Status do projeto
├── requirements.txt # NOVO · libs ainda não usadas pelo código atual
├── auto_scan.py
├── install.sh
├── scan.sh
├── setup.sh
└── update.sh
```

Itens marcados como "NOVO" foram adicionados ou criados desde a versão anterior desta Bíblia (manhã de 20/06/2026).

04 Pré-requisitos

Obrigatórios

- **Python 3.8+** — download em python.org
- **Git** (para clonar o repositório)

Verificando instalação

```
# Verifica Python
python3 --version
# Saída esperada: Python 3.8.x ou superior

# Verifica Git
git --version
# Saída esperada: git version 2.x.x
```

Dependências do projeto

`requirements.txt` foi expandido recentemente, mas hoje **nenhum módulo em uso depende delas** — `ping_sweep`, `port_scanner`, `handler.py` e `logger.py` usam só biblioteca padrão do Python. Elas são pré-requisito apenas se você for usar a exportação HTML (`jinja2`) ou desenvolver os módulos futuros (`dnspython`, `cryptography` etc.).

```
pip3 install -r requirements.txt
```

Sistema operacional

- ✓ Linux (Ubuntu, Debian, Fedora, etc.)
- ✓ Windows (10, 11 com WSL ou PowerShell)
- ✓ macOS (10.15+)

05 Formas de execução

5.1 CLI nova — `list` / `run` (oficial)

Caminho que o `main.py` atual implementa. Use sempre o subcomando antes do nome do módulo.

```
python3 main.py list
python3 main.py run <módulo> --target <alvo> [--verbose] [--output-file arquivo] [--format json|html]
```

Listar módulos (funciona hoje):

```
python3 main.py list
```

```
Módulos disponíveis:
- ping_sweep      - Verifica hosts ativos via ICMP
- port_scanner    - Escaneia portas TCP abertas
- dns_enum        - Enumera registros DNS
- ssl_scan        - Analisa certificados SSL/TLS
- http_headers    - Analisa headers HTTP de segurança
- subdomain_brute - Descobre subdomínios por brute force
- traceroute      - Rastreia a rota até o alvo
```

Os 5 últimos módulos da lista ainda não existem no código — aparecem aqui porque a lista está fixa no texto de `main.py`, não é gerada dinamicamente.

Executar um módulo (hoje quebra — ver Bug 1):

```
python3 main.py run ping_sweep --target 8.8.8.8 --verbose
```

▲ RESULTADO ATUAL

`AttributeError: 'NoneType' object has no attribute 'items'` — o módulo imprime o resultado na tela e retorna `None`; o `run_module()` de `main.py` tenta iterar sobre esse retorno como se fosse um dicionário. Use a forma legada (5.2) até isso ser corrigido.

5.2 Forma legada — chamando o módulo direto (funciona hoje)

Enquanto o Bug 1 não é corrigido, esta é a forma que executa `ping_sweep` e `port_scanner` sem erros, chamando a função `run()` do módulo diretamente via Python:

```
python3 -c "from beaversec.modules.ping_sweep import run; run('8.8.8.8', verbose=True)"
```

Ou criando um miniscript de uma linha:

```
# scan_direto.py
from beaversec.modules.ping_sweep import run
run('192.168.1.0/24', verbose=True)
```

```
python3 scan_direto.py
```

Os próprios módulos têm um bloco de teste embutido, então também funcionam executados diretamente:

```
python3 beaversec/modules/ping_sweep.py
python3 beaversec/modules/port_scanner.py
```

5.3 Instalação global do comando `beaver`

```
# Cria link simbólico no sistema
sudo ln -s $(pwd)/main.py /usr/local/bin/beaver

# Agora pode executar de QUALQUER lugar!
beaver list
beaver run ping_sweep --target 8.8.8.8
```

Desinstalar:

```
sudo rm /usr/local/bin/beaver
```


06 Comandos básicos

CLI nova (main.py atual)

Comando	Descrição	Exemplo
list	Lista os módulos disponíveis	python3 main.py list
run <módulo> --target <alvo>	Executa um módulo BUG 1	python3 main.py run port_scanner --target 8.8.8.8
--verbose	Ativa modo detalhado	... run ping_sweep --target 8.8.8.8 --verbose
--output-file <arquivo>	Salva resultado em arquivo	... --output-file relatorio.json
--format json html	Formato do relatório (html tem BUG 2)	... --format json
-v, --version	Mostra a versão do BeaverSec	python3 main.py --version
-h, --help	Mostra ajuda completa	python3 main.py --help

Estrutura do comando novo

```
python3 main.py run [MÓDULO] --target [ALVO] [OPÇÕES]
                    |           |           |
                    |           |           | --verbose, --format, --output-file
                    |           |           | IP, domínio ou URL
                    |           |           |
                    |           |           | nome do módulo
```

Forma legada (chamada direta — funciona hoje)

Uso	Exemplo
Importar e chamar run()	python3 -c "from beaversec.modules.ping_sweep import run; run('8.8.8.8')"
Executar o arquivo do módulo direto	python3 beaversec/modules/ping_sweep.py

07 Módulos disponíveis

7.1 ping_sweep IMPLEMENTADO — Varredura ICMP

Descrição: descobre hosts ativos usando ping (ICMP Echo Request). Multi-threaded (até 100 workers simultâneos).

Tipos de alvo:

Tipo	Formato	Exemplo
IP único	X.X.X.X	8.8.8.8
Rede CIDR	X.X.X.X/Y	192.168.1.0/24

Uso (forma legada, funciona hoje):

```
python3 -c "from beaversec.modules.ping_sweep import run; run('8.8.8.8', verbose=True)"
python3 -c "from beaversec.modules.ping_sweep import run; run('192.168.1.0/24')"
```

Saída esperada:

```
✓ 192.168.1.1 está ATIVO
✓ 192.168.1.10 está ATIVO
✓ 192.168.1.11 está ATIVO
```

```
=====
✓ SCAN CONCLUÍDO COM SUCESSO!
  Total de hosts ativos: 3
```

```
LISTA DE HOSTS ATIVOS:
  1. 192.168.1.1
  2. 192.168.1.10
  3. 192.168.1.11
=====
```

7.2 port_scanner IMPLEMENTADO — Varredura de portas

Descrição: varre 23 portas TCP comuns (FTP, SSH, HTTP, HTTPS, SMB, RDP, MySQL, PostgreSQL, MongoDB, Redis etc.) usando sockets, multi-threaded (50 workers).

Uso (forma legada, funciona hoje):

```
python3 -c "from beaversec.modules.port_scanner import run; run('scanme.nmap.org', verbose=True)"
```

Saída esperada:

```
✓ Porta 22 ABERTA (SSH)
✓ Porta 80 ABERTA (HTTP)
```

```
=====
✓ SCAN CONCLUÍDO!
  Total de portas abertas: 2
```

```
PORTAS ABERTAS:
  22 - SSH
  80 - HTTP
=====
```

7.3 meu_modulo IMPLEMENTADO — Template de exemplo

Descrição: módulo de exemplo para quem vai criar uma ferramenta nova. Não faz varredura real.

```
python3 -c "from beaversec.modules.meu_modulo import run; run('8.8.8.8', verbose=True)"
```

7.4 Módulos planejados AINDA NÃO EXISTEM

Aparecem em `python3 main.py list` e em `tests/test_modules.py` , mas os arquivos ainda não foram criados em `beaversec/modules/` :

Módulo	Função prevista
<code>dns_enum</code>	Enumera registros DNS
<code>ssl_scan</code>	Analisa certificados SSL/TLS
<code>http_headers</code>	Analisa headers HTTP de segurança
<code>subdomain_brute</code>	Descobre subdomínios por brute force
<code>traceroute</code>	Rastreia a rota até o alvo

7.5 Criando seu próprio módulo

1 Crie um arquivo em `beaversec/modules/`

```
# beaversec/modules/meu_modulo.py
def run(target: str, verbose: bool = False) -> None:
    """Função obrigatória para o módulo"""
    print(f"Executando meu módulo contra {target}")
    if verbose:
        print("Modo detalhado ativado")
```

⚠ **ATENÇÃO SE FOR USAR COM `main.py run`**

Para o subcomando novo `run` funcionar sem cair no Bug 1, sua função precisa **retornar um dicionário** em vez de `None` , por exemplo `return {"status": "ok", "target": target}` .

2 Execute seu módulo (forma legada)

```
python3 -c "from beaversec.modules.meu_modulo import run; run('8.8.8.8', verbose=True)"
```

08 Erros comuns e soluções

Erro 1: AttributeError: 'NoneType' object has no attribute 'items'

✗ ERRO

Acontece com `python3 main.py run <módulo> --target <alvo>` (Bug 1, ver seção 02)

✓ SOLUÇÃO

Use a forma legada até o módulo retornar um dicionário

```
# X - QUEBRA HOJE
python3 main.py run ping_sweep --target 8.8.8.8

# OK - CORRETO
python3 -c "from beaversec.modules.ping_sweep import run; run('8.8.8.8')"
```

Erro 2: ModuleNotFoundError: No module named 'beaversec.exporters'

✗ ERRO

Acontece com `--format html` (Bug 2). O arquivo existe em `exporters/`, não em `beaversec/exporters/`

✓ SOLUÇÃO

Use `--format json` por enquanto, ou mova a pasta `exporters/` para dentro de `beaversec/`

```
# X - QUEBRA HOJE
python3 main.py run ping_sweep --target 8.8.8.8 --format html

# OK - CONTORNA O PROBLEMA
python3 main.py run ping_sweep --target 8.8.8.8 --format json
```

Erro 3: Comando 'python' não encontrado

✗ ERRO

Comando 'python' não encontrado, você quis dizer:
comando 'python3' do deb python3

✓ SOLUÇÃO

Use `python3` no lugar de `python`

```
python3 main.py list # OK - CORRETO
```

Erro 4: Permission denied

✗ ERRO

```
./main.py list
bash: ./main.py: Permission denied
```

✓ SOLUÇÃO

Dê permissão de execução

```
chmod +x main.py
./main.py list # OK - FUNCIONA
```

Erro 5: ModuleNotFoundError: No module named 'beaversec'

✗ ERRO

ModuleNotFoundError: No module named 'beaversec'

✓ SOLUÇÃO

Execute da raiz do projeto

```
cd ~/BEAVERSEC
python3 main.py list # OK - CORRETO
```

Erro 6: NENHUM HOST ATIVO ENCONTRADO

✗ ERRO

NENHUM HOST ATIVO ENCONTRADO.

Causas e soluções:

- **Alvo incorreto:** verifique o IP ou CIDR
- **Sem conexão:** teste com `ping 8.8.8.8`
- **Firewall:** pode estar bloqueando ICMP
- **Fora da rede:** verifique se está na mesma rede

Erro 7: CIDR inválido

✗ ERRO

ValueError: CIDR inválido

✓ SOLUÇÃO

Use o formato correto

```
# X - ERRADO
python3 -c "from beaversec.modules.ping_sweep import run; run('192.168.1/24')"
```

```
# OK - CORRETO
python3 -c "from beaversec.modules.ping_sweep import run; run('192.168.1.0/24')"
```

Erro 8: ModuleNotFoundError ao rodar os testes

✗ ERRO

ModuleNotFoundError: No module named 'beaversec.utils.security' (Bug 3)

✓ SOLUÇÃO

Crie `beaversec/utils/security.py` com as funções `validate_ip`, `validate_domain`, `validate_cidr`, `sanitize_target` e `validate_target`, ou remova esse teste até o módulo existir

09 Scripts prontos para uso

► ATUALIZADO NESTA VERSÃO

Os scripts abaixo foram ajustados para a sintaxe nova do `main.py` (`list` / `run --target`). As versões anteriores chamavam `python3 main.py ping_sweep <alvo>` direto, que não existe mais nesse `main.py`.

setup.sh

9.1 Configuração rápida

```
#!/bin/bash
echo "Configurando BeaverSec..."

chmod +x main.py
echo "[OK] main.py agora é executável"

if ! grep -q "alias beaver=" ~/.bashrc; then
    echo "alias beaver='python3 $(pwd)/main.py'" >> ~/.bashrc
    echo "[OK] Alias 'beaver' adicionado ao .bashrc"
fi

read -p "[?] Deseja instalar globalmente? (s/N) " -n 1 -r
echo
if [[ $REPLY =~ ^[Ss]$ ]]; then
    sudo ln -sf $(pwd)/main.py /usr/local/bin/beaver
    echo "[OK] BeaverSec instalado globalmente!"
fi

echo ""
echo "Configuração concluída!"
echo "Agora você pode usar:"
echo "    beaver list"
echo "    beaver run ping_sweep --target 8.8.8.8"
```

Como usar:

```
chmod +x setup.sh
./setup.sh
```

scan.sh

9.2 Varredura rápida

```
#!/bin/bash
echo "BeaverSec - Scanner Rápido"

if [ $# -eq 0 ]; then
    echo "Uso: ./scan.sh "
    echo "    ./scan.sh 8.8.8.8"
    echo "    ./scan.sh 192.168.1.0/24"
    exit 1
fi

python3 -c "from beaversec.modules.ping_sweep import run; run('$1', verbose=True)"
```

Usa a forma legada de chamada (5.2) porque `main.py run` ainda quebra com `ping_sweep` (Bug 1).

Como usar:

```
chmod +x scan.sh
./scan.sh 8.8.8.8
./scan.sh 192.168.1.0/24
```

auto_scan.py

9.3 Varredura automática

```
#!/usr/bin/env python3
from beaversec.modules.ping_sweep import run
import time

targets = [
    "8.8.8.8",
    "1.1.1.1",
    "192.168.1.0/24",
]

for target in targets:
    print(f"\nEscaneando: {target}")
    run(target, verbose=True)
    time.sleep(1)

print("\n[OK] Todas as varreduras concluídas!")
```

Como usar:

```
chmod +x auto_scan.py
python3 auto_scan.py
```

install.sh

9.4 Instalação de dependências

```
#!/bin/bash
echo "BeaverSec - Instalando dependências..."

if ! command -v python3 &> /dev/null; then
    echo "[ERR0] Python 3 não encontrado!"
    exit 1
fi

echo "[OK] Python 3 encontrado: $(python3 --version)"

if [ -f "requirements.txt" ]; then
    pip3 install -r requirements.txt
fi

chmod +x main.py
echo "[OK] Instalação concluída!"
```

update.sh

9.5 Atualização do GitHub

```
#!/bin/bash
echo "Atualizando BeaverSec..."

git pull origin main
chmod +x main.py

echo "[OK] BeaverSec atualizado!"
echo "Para ver as novidades: python3 main.py list"
```

10 Exemplos práticos

Exemplo 1: descobrindo hosts na rede local

```
python3 -c "from beaversec.modules.ping_sweep import run; run('192.168.1.0/24', verbose=True)"
```

Saída:

```
✓ 192.168.1.1 está ATIVO
✓ 192.168.1.10 está ATIVO
✓ 192.168.1.11 está ATIVO
```

```
LISTA DE HOSTS ATIVOS:
1. 192.168.1.1
2. 192.168.1.10
3. 192.168.1.11
```

Exemplo 2: verificando se um servidor está online

```
python3 -c "from beaversec.modules.ping_sweep import run; run('8.8.8.8', verbose=True)"
```

Exemplo 3: varrendo portas de um host

```
python3 -c "from beaversec.modules.port_scanner import run; run('scanme.nmap.org', verbose=True)"
```

Exemplo 4: listando módulos disponíveis

```
python3 main.py list
```

Exemplo 5: usando no Windows (PowerShell)

```
python main.py list
python -c "from beaversec.modules.ping_sweep import run; run('8.8.8.8')"
```

Exemplo 6: criando um alias permanente

Linux/macOS:

```
echo "alias beaver='python3 /home/usuario/BEAVERSEC/main.py'" >> ~/.bashrc
source ~/.bashrc
beaver list
```


11 Lista de comandos por tarefa

Referência rápida: encontre a tarefa que você quer fazer e copie o comando direto para o terminal. Comandos marcados `$` rodam no bash/PowerShell; os marcados `py` rodam dentro do Python ou via `python3 -c`.

Para listar os módulos disponíveis

```
$ python3 main.py list
$ beaver list (após instalação global)
```

Para descobrir hosts ativos numa rede (ping sweep)

```
py python3 -c "from beaversec.modules.ping_sweep import run; run('192.168.1.0/24')"
$ ./scan.sh 192.168.1.0/24
```

Para verificar se um único IP está online

```
py python3 -c "from beaversec.modules.ping_sweep import run; run('8.8.8.8', verbose=True)"
$ ./scan.sh 8.8.8.8
```

Para escanear portas TCP abertas de um host

```
py python3 -c "from beaversec.modules.port_scanner import run; run('scanme.nmap.org')"
py python3 beaversec/modules/port_scanner.py
```

Para rodar um módulo em modo detalhado (verbose)

```
py python3 -c "from beaversec.modules.ping_sweep import run; run('8.8.8.8', verbose=True)"
```

Para deixar o main.py executável direto (sem digitar python3)

```
$ chmod +x main.py
$ ./main.py list
```

Para instalar o comando beaver globalmente

```
$ sudo ln -s $(pwd)/main.py /usr/local/bin/beaver
$ beaver list
```

Para desinstalar o comando beaver

```
$ sudo rm /usr/local/bin/beaver
```

Para criar um alias permanente (usar "python" em vez de "python3")

```
$ echo "alias python='python3'" >> ~/.bashrc
$ source ~/.bashrc
```

Para instalar as dependências do projeto

```
$ pip3 install -r requirements.txt
$ ./install.sh
```

Para atualizar o projeto com as últimas mudanças do GitHub

```
$ git pull origin main
$ ./update.sh
```

Para criar um módulo novo

```
$ nano beaversec/modules/meu_modulo.py
py python3 -c "from beaversec.modules.meu_modulo import run; run('8.8.8.8', verbose=True)"
```

Para checar se Python e Git estão instalados

```
$ python3 --version
$ git --version
```

Para rodar a varredura automática em múltiplos alvos

```
$ chmod +x auto_scan.py
$ python3 auto_scan.py
```

12 Perguntas frequentes

Q1: Por que `python3 main.py run ping_sweep --target 8.8.8.8` dá erro?

É o Bug 1 (seção 02): o módulo retorna `None` e o subcomando `run` espera um dicionário. Use a forma legada: `python3 -c "from beaversec.modules.ping_sweep import run; run('8.8.8.8')"`.

Q2: Por que o comando `python` não funciona?

No Linux/macOS, o comando padrão é `python3`. Use `python3` ou crie um alias.

Q3: Como instalar o BeaverSec?

Não precisa instalar! Basta clonar o repositório e executar.

Q4: Posso usar no Windows?

Sim! Use no PowerShell com `python` ou no WSL com `python3`.

Q5: Como adicionar um novo módulo?

Crie um arquivo em `beaversec/modules/` com uma função `run()`. Se quiser usá-lo com `main.py run`, faça-a retornar um dicionário.

Q6: Por que não encontro hosts ativos?

Verifique se está na mesma rede, se o firewall não bloqueia ICMP e se o alvo está correto.

Q7: O BeaverSec precisa de privilégios root?

Não, pode ser executado como usuário normal.

Q8: Como atualizar o BeaverSec?

Use `git pull origin main` na pasta do projeto, ou rode `./update.sh`.

Q9: Por que `python3 main.py list` mostra módulos que não existem?

A lista em `main.py` está escrita fixa no código (não é gerada a partir dos arquivos reais), e inclui módulos planejados que ainda não foram implementados. Ver seção 07.4.

Q10: Como desinstalar a instalação global?

```
sudo rm /usr/local/bin/beaver
```

13 Contribuindo

Prioridades atuais (ajude a corrigir)

- **BUG 1** Fazer `ping_sweep` e `port_scanner` retornarem um dicionário em vez de `None`, para compatibilizar com `main.py run`
- **BUG 2** Mover `exporters/` para dentro de `beaversec/exporters/` (ou corrigir o import em `main.py`)
- **BUG 3** Criar `beaversec/utils/security.py` com as funções esperadas pelos testes
- **NOVO MÓDULO** Implementar `dns_enum`, `ssl_scan`, `http_headers`, `subdomain_brute` ou `traceroute`

Como adicionar um módulo

- 1 Crie o arquivo em `beaversec/modules/`:

```
# beaversec/modules/meu_modulo.py
def run(target: str, verbose: bool = False) -> None:
    print(f"Executando contra: {target}")
```

- 2 Teste com a forma legada:

```
python3 -c "from beaversec.modules.meu_modulo import run; run('8.8.8.8', verbose=True)"
```

Como reportar bugs

Acesse: github.com/ojhonatanls/beaversec/issues

Como sugerir melhorias

Faça um fork, crie uma branch, commit e abra um Pull Request.

Links úteis

Repositório: github.com/ojhonatanls/beaversec

Issues: github.com/ojhonatanls/beaversec/issues

Python: python.org



*“Conhecimento compartilhado é
conhecimento multiplicado.”*

BeaverSec – Binary Enumeration & Advanced Vulnerability Exploitation Recon
© 2026 BeaverSec Team. Todos os direitos reservados.